

MILTANK — the search player

Named for the classic Rollout user. `engine/rollout_leaf.js` is the mechanism; MILTANK is the player that uses it.

1. What it is, and why it is not MAG

ABRA has two players and until 2026-08-03 only one of them had a name.

MAG (`engine/magnemite.js`) is a fitted linear policy over `board.js` features, trained to predict **what a human clicks**. It is an imitator. Given a position it scores every legal option against a weight vector fitted on 60,000 corpus decisions and takes the best — one step, no lookahead, no notion of what happens next.

MILTANK decides by **playing the position out**. It forces a candidate pair of clicks, lets the game run to a conclusion, and takes the line that wins most. It has no fitted weights of its own and no opinion about what a person would do. It can therefore prefer a move that appears nowhere in the corpus, and it routinely does.

They are not rivals and MILTANK is not a replacement. The shipped arrangement is:

*MILTANK searches. When the search **cannot separate its options** — the finalists fall inside one standard error, or the position is already decided — it hands the decision back to MAG.*

That fallback is the point rather than a hedge. An argmax over estimates that overlap is a coin flip wearing a search's clothes, and a fitted human prior is a better tiebreak than dice. **Imitation is the floor; search is the ceiling.**

2. Where MILTANK decides

Decision	Before	Now
Which 4 to bring, which 2 to lead	<code>return 'default'</code> — packed order	90 brings enumerated and played out
Both clicks each turn	MAG's argmax	successive halving over every legal pair
Post-KO replacement	one-step heuristic	each candidate played out

The preview one matters most. Showdown's `RandomPlayerAI.chooseTeamPreview` returns the literal string `default`, `magnemite` never overrode it, so **every game ever played against this bot had its lead decided by where a Pokémon happened to sit in a team string** — on what `prior_player.js` itself calls *"the single largest branch in the game"*.

All three now call the same leaf. Until 2026-08-04 the preview did not: it held its own playout loop, calling `battleInit` / `battleTurn` directly with deterministic greedy on both sides, and never reached `rolloutWinProb` at all. MILTANK therefore shipped **two players**, and only the in-game one was ever swept — so the measured 53.22%-preview against 50.99%-in-game contrast was partly a contrast between two *implementations* rather than between two settings of one. `rollout_leaf.runPlayout` is the single implementation now and the differences are parameters. Greedy is still reachable; it is `previewExplore: 0`. See [SEARCH.md](#) open item 0 for exactly what moved.

The preview also seeded a game that had not started. It passed `battleInit({seeded: true})`, which exists to stop a *mid-battle* leaf re-firing entry effects that already happened. At preview nobody has entered yet, so it deleted the whole switch-in class from the one decision they matter most for: no Intimidate, no

Drought, no Drizzle, no Sand Stream, no Snow Warning, no terrain setter on turn one. Deciding a lead is largely deciding who eats an Intimidate and the search could not see one. `seeded` is a parameter of `rolloutWinProb` now, default unchanged, and preview passes `false`.

3. The academic position, and where MILTANK sits in it

3.1 This is a simultaneous-move game, and MILTANK does not solve it

Pokémon is not turn-based. Both players commit without seeing the other, which puts it in the **simultaneous-move** class where a deterministic best response is exploitable by construction — the matrix game at each node can have no pure equilibrium.

The literature offers several MCTS variants for this setting: **Decoupled UCT (DUCT)**, sequential UCT, Exp3, and regret matching (Lanctot et al., CIG 2014; Tak, Lanctot & Winands). DUCT lets each player pick from their own statistics without modelling the other. It is known **not to converge to an optimal strategy even in a single-state game** — and it still performs best empirically across their test set.

MILTANK is weaker than all of these and the docs should say so. It computes a *best response to a fixed opponent policy*: the opponent plays `chooseAction` during the stepped turn, identically for every candidate. That makes the comparison across candidates like-for-like, which is what the measurement needs, but it is not an equilibrium and it is exploitable by anyone who models it.

It is the right first cut for one stated reason: if a best response does not diverge from MAG, a mixture over the same cells will not either. `data/rollout-r3.json` records 72.9% divergence on 70 decisions, so it does diverge, and the equilibrium version is worth building.

3.2 The playout is fully random, and that is deliberate

The single most counter-intuitive result in this literature is that **a stronger playout policy can make the search worse**. Policies that maximise MCTS performance as playouts are *weak* at playing the game (Are Strong Policies Also Good Playout Policies?, AIIDE; An Analysis of Monte Carlo Tree Search). What matters is how a policy *rank*s actions relative to each other, and **low-variance policies are dangerous** — a deterministic greedy playout replays the same line from one position, so N samples carry barely more information than one.

ABRA measured exactly this pathology before reading the literature. `chooseAction` is deterministic greedy; 53% of positions landed in the 0–10% or 90–100% bin and those bins were wrong by 22–29 points, and accuracy was **flat in N**. R1 then found a fully random playout judges a position at **68.18%** against **64.42%** for the greedy one.

That comparison was withdrawn on 2026-08-04 as UNCHECKABLE. The 68.18% was never written to an artifact — `engine/rollout_r1.js` printed it — and the one committed row dump turns out to hold the *greedy* column, so the figure could not be recomputed from anything in the repository.

`data/rollout-r1.json` records what survived, the greedy arm, at **65.72% against material's 65.26%, +0.46 [-0.72, +1.63] — UNDECIDED.**

It was re-run the same day and it reproduces. `data/rollout-r1-explore-sweep.json`, from a three-arm sweep over the identical 9,201 positions:

explore	accuracy	Brier	log-loss	ECE	saturated
o (greedy)	65.72%	0.259	1.822	0.196	50.7%
0.5	67.58%	0.222	1.028	—	—
1.0	67.97%	0.213	0.863	0.104	29.4%
material	65.27%	0.213	0.612	0.050	—

Paired: **+2.25 points, 95% CI [1.31, 3.19]** for 1.0 over greedy, monotone in explore, and it holds at the live 60-turn horizon (67.46% against 64.21% on a second sample). The published 68.18% lands at 67.97% and the published +2.91 over material at +2.71 [1.60, 3.82].

So the retraction was right about the provenance and the claim survives it. `--rollout-explore = 1.0` **stands, and it now stands on an artifact**. Two corrections that came with it: the "64.42% for greedy" quoted in `rollout_leaf.js:147` and `mag_bot.js:145` does *not* reproduce — greedy is 65.7% on both the committed dump and a fresh run — so those comments overstate the gap and should cite the sweep artifact instead. And the saturation story is confirmed rather than merely argued: the greedy payout puts **half** its positions in the two extreme bins and `explore=1.0` puts under a third, halving the calibration error. A deterministic payout replays one line, exactly as §3.2 claims.

Caveat, measured 2026-08-03: that result is about *judging a position*. Judging and *choosing an action* are different jobs, and a random opponent never punishes a wasted turn. A direct test on 40 corpus positions found Protect scoring -4.3pt at `explore 1.0` and -5.2pt at `0.0` — i.e. the randomness is **not** obviously overpricing tempo. That is one measurement on one mechanic and it should not be read as settling the question.

3.3 The picker is a fixed-budget best-arm identification problem

A turn offers up to ~90 legal pairs. Estimating each and taking the max is **not** the same as finding the best one: the max over K noisy estimates is biased upward by roughly the spread of the noise, so the winner is disproportionately the arm that got lucky. At $n=120$ a win rate near 0.7 carries a standard error around 4 points, and the max over 63 such arms is inflated by ~2 SE of pure dice — larger than most real differences between two reasonable clicks.

The symptom was visible in play: **the bot clicked Recover at full HP**, a move MEDICHAM correctly heals o with. It was not choosing Recover; it was choosing whichever arm rolled well.

This is textbook **fixed-budget best-arm identification**, and the standard remedy is **successive halving**: spend the budget where it discriminates, not uniformly ([sequential halving](#) / [fixed-budget BAI](#)). MILTANK screens every pair at $N/3$, keeps the top 8, and re-tests those at $2N$ **with fresh seeds** — so an arm that advanced on lucky dice has to roll them twice. It is also cheaper than the flat version it replaced: $63 \times 40 + 8 \times 240$ beats 63×120 .

3.4 On-line policy improvement

The whole shape — take a base policy, improve it at decision time by rolling out — is Tesauro and Galperin's on-line policy improvement ([arXiv:2501.05407](#)). MAG is the base policy; MILTANK is the improvement operator. That framing predicts the fallback behaviour: where the rollout has no signal, the improved policy should return the base policy, which is what the noise-floor check does.

3.5 What the field says about the domain

[VGC-Bench](#) puts the team-configuration space at $\sim 10^{139}$, larger than Chess, Go, Poker, StarCraft or Dota, and ships 700,000 human battle logs with heuristic, LLM, behaviour-cloning and multi-agent-RL baselines. Two findings bear directly on ABRA:

1. In **single-team mirror matches** their methods beat a professional VGC competitor. Strength in a narrow team distribution is achievable; that is roughly MILTANK's situation, playing a fixed pool of 230 open-sheet teams.
2. The agent that is best on **one** team is **more exploitable and generalises worse** across teams. This is a warning aimed squarely at ABRA: a bot tuned against the 230-team pool should not be assumed to hold up outside it, and any strength claim has to name the team distribution it was measured on.

3.6 The seeded leaf assumed the mega and forgot the weather — fixed 2026-08-04

`dmgMon` builds a stone-holder with the **mega's** ability: a Charizard carrying Charizardite Y arrives in the playout with `drought`, not `blaze`. `battleInit({seeded:true})` then suppresses entry effects, correctly — the actives are already standing there and re-firing Intimidate would drop the same Attack twice — so the sun that ability implies never fired. `applyMegaWeather` existed to close exactly that gap and **its write was discarded by the very next line** for as long as it existed, so Mega Charizard Y stood in clear weather in every mid-battle rollout this project ever ran.

Fixed by applying the caller's field first and the mega's weather second, so the function's own guard arbitrates. **It is now mega-gated, and that gate is the load-bearing half:** the discarded version had no mega check, and un-discarding it without one would have invented a Torkoal's sun on a board the tracker says is clear. The board is authoritative for a body that is what the board says it is; a mega is the one case where the rollout's body is something else.

Measured, in one process so the engine cannot differ between arms: **15 of 250 corpus boards moved, every one of them a mega weather setter standing in no weather, and nothing else moved at all.** Charizard-Mega-Y's sun is worth **+11.0 pt to the side that owns it and -12.5 pt to the side facing it.** Full parity table, the direct playout counter, and the much larger weather defect this uncovered are in [SEARCH.md](#) items 5 and 5b.

Every leaf number in this document predates it. R1, R2, R3, R4 and the calibration were all computed with the mega's weather missing.

3.7 The search spoke a third dialect of terrain — fixed 2026-08-04

Same shape as 3.6 and worth stating beside it, because it is the second time a field the leaf was *given* never reached the engine. ENGINE routed every terrain read in `medicham2-browser.js` through `terrainId()` — `weatherId()`'s sibling — and then measured that **0 of 863 terrain-carrying boards reached the leaf at all.** The cause was upstream of the engine entirely: `miltank.js` and `rollout_r1.js` built their field object with

```
terrain: ['electric', 'grassy', 'misty', 'psychic'].find(t => board.hasField(t)) || ''
```

Those are the **engine's** words. `board.startField` stores the dex's `move.terrain`, which is `electricterrain`, `grassyterrain`, `mistyterrain`, `psychicterrain`. So a third spelling was being probed against a Board that has never held it, it matched nothing on every board ever walked, and the field handed to every rollout carried `terrain: ''` whether or not a terrain was up. Terrain was correct in the engine and invisible to the search.

Fixed by one helper, `rollout_leaf.terrainOnBoard`, which probes the board's own keys and translates with `MEDI.terrainId`. **No fourth map was written**, and the post-KO replacement search — which had `terrain: ''` hardcoded — now reads the board too.

The honest size of it, measured rather than claimed: near zero, and that was the expected answer. On 3,256 corpus boards, 29 carry a terrain (0.9%; ENGINE's whole-corpus figure is 1.24%) and the old probe found 0 of them. Rolling out all 29 plus 176 terrain-free controls at n=40, **4 boards moved, all four Psychic Terrain, mean |Δ| 8.75 pt, and 0 of 176 controls moved.** Every Electric Terrain board was unchanged, because the engine's terrain readers are thin: the priority block, Grassy Glide's priority, Hadron Engine and the two `terrainScaled` moves. A rare condition times a thin reader is a small number, and reporting it as one is the point.

4. Open team sheets are what make the preview search possible

MILTANK's bring/lead search is only meaningful because `|showteam|` hands over **the opponent's entire team with full sets**. A fitted bring prior can say "*Whimsicott leads 53% of the time*" — a fact about the population, not about this game. The sheet is information no usage prior can encode.

The opponent's own bring is **marginalised, not assumed**: each playout samples one of their 15 possible fours. Fixing a guess would optimise against one opponent out of fifteen and call it a plan.

Known weakness: that sample is *uniform*. A real opponent is not equally likely to bring every four. Weighting it by `bring_priors` or by which four best answers MILTANK's own team is unbuilt.

The sample is now shared across brings. Each candidate bring used to be scored against its own independently seeded opponent draws, so the difference between two brings sat underneath that noise; one seed now serves the whole preview, so playout *i* of every bring faces the same four and the same dice. Common random numbers, the identical fix the post-KO replacement search needed for the identical reason — there, five of five candidates fell inside their own error bar and the search deferred to MAG on every replacement of a live game.

And the enumerator was wrong whenever a body would not build. It mixed positions in the buildable list with team indices; on a six-mon team with two unbuildable bodies it produced 19 brings where 6 exist, 18 of which named a Pokémon it had just declared unbuildable, and scored three-mon brings as four-mon ones. Full teams enumerated 90 correctly, which is why it survived. Fixed 2026-08-04.

5. What MILTANK is not

- **Not an equilibrium.** Best response to a fixed policy (§3.1). Exploitable by design.
- **Not a tree search.** One step of forced actions, then a playout. No selection, no backprop, no tree. Calling it MCTS would overclaim.
- **Not measured to be stronger than MAG.** R1 (leaf accuracy), R2 (cost), R3 (divergence) all pass. **R4 — the SPRT against shipped greedy — has never been run.** Every claim in this document is about mechanism, not about winning more games. As of 2026-08-03 the bot's record against its author is 0–8.

6. Files

File	Role
<code>engine/miltank.js</code>	the player — the three decision points, installed onto a magnemite instance
<code>engine/rollout_leaf.js</code>	<code>rolloutWinProb</code> , <code>rolloutAfterActions</code> — the leaf; <code>runPlayout</code> is the playout
<code>engine/mag_bot.js</code>	the live wiring and the CLI flags; <code>--miltank</code> (<code>--rollout</code> aliases it)
<code>engine/medicham2-browser.js</code>	the doubles engine the playouts run in
<code>engine/rollout_r1/r2/r3.js</code>	the gates. R5, the action-ranking backtest, is specified in SEARCH.md and unbuilt
<code>docs/ROLLOUT-design.md</code>	the original design note

Flags

Flag	Default	Meaning
<code>--miltank</code>	off	use the search player
<code>--rollout-n</code>	200	playouts per candidate at the final stage
<code>--rollout-explore</code>	1.0	playout randomness, now for the preview too . Re-earned 2026-08-04, <code>data/rollout-r1-explore-sweep.json</code> : +2.25 [1.31, 3.19] over greedy as a

Flag	Default	Meaning
		JUDGE. Not measured as a PLAYER — <code>mew.js</code> has no <code>--miltank-explore</code> (PRIORITIES #33)
<code>--rollout-turns</code>	60	horizon before a playout is scored
<code>--preview-n</code>	40	playouts per candidate bring
<code>--preview-ms</code>	15000	preview deadline; whatever was scored by then is reported
<code>--miltank-foe</code>	uniform	the playout opponent: a coin flip, or <code>prior</code> — what the species really clicks. Now reaches the preview ; before the unification the preview ignored it
<code>MILTANK_TIMING=<path></code>	off	write the per-decision wall-clock artifact. Reduce with <code>node engine/miltank.js -reduce <path></code>
<code>MILTANK_CLOCK=1</code>	off	adaptive spend against the bank. Env rather than flag because <code>mew.js</code> and <code>mag_bot.js</code> are other divisions' files
<code>MILTANK_EARLY_DEFER=1</code>	off	skip the finalist round when the screen says the position is heading for the tie band. Changes what is clicked — its own SPRT arm, not a timing claim

One option has **no flag yet**: `previewExplore` (`DEFAULTS.previewExplore` , null = follow `explore`) is the knob that runs the preview greedy again. Wiring it needs a one-line `arg()` in `mag_bot.js` and `mew.js` , neither of which is SEARCH's file — the same one-liner PRIORITIES #33 already owes `--miltank-explore` .

The budget — and the timer has never been on in a game anyone has watched

`budgetMs` is 20,000 and `previewMs` is 15,000. Our format's VGC Timer (`data/rulesets.ts:778`) is `Timer Starting = 420` , `Timer Grace = 90` , `Timer Add Per Turn = 0` , `Timer Max Per Turn = 55` , `Timer Max First Turn = 90` , `Timeout Auto Choose` , `DC Timer Bank` — one drawn bank, no refill, and the per-turn cap is `Math.min(bank, 55)` so the two are **one clock**.

The bank is 420 s, not 510, and this file said 510. `room-battle.ts:209` does start at `starting + grace = 510` , but `updateTurn ( :305-306 )` re-imposes `secondsLeft = Math.min(secondsLeft + 0, starting)` on **every new turn** and `starting` is 420. The grace is **use-it-or-lose-it on the first timed request** and is not bankable. So `budgetMs: 20000` buys **21 decisions**, not the 24 this file claimed, with preview free because it comes out of grace that is about to be clamped away regardless.

And the bank does not tick while the timer is off. `secondsLeft` falls only in `nextTick ( :353 )`, scheduled only by `nextRequest` , which returns at `:320` when nobody has requested the timer.

`Config.forcetimer` is off on the main server, so the timer is **per player and can be switched on mid-game** — Will, 2026-08-04: *"when i play the bot i have turned the timer off"*, which is why a constant has always looked fine. An opponent typing `/timer on` at turn 9 nevertheless finds a **full** bank, not one MILTANK has already spent. The mid-game switch-on is benign.

MILTANK can see the clock exactly and does not read it. `:332` privately sends the player `|inactive|Time left: X sec this turn | Y sec total | Z sec grace` on every request while the timer is on (the real bank is `Y + Z` ; `:330` subtracts the grace out of the total). `mag_bot.js` parses **zero** `|inactive|` lines. `bot.noteClock()` and the counter `bot.clockStats().notes` now exist here for OPS to wire; **that counter is o today**, and the adaptive rule is therefore designed to be correct without it.

Adaptive spend is implemented, behind `clock` , DEFAULT OFF — `(bank - reserve) / expected remaining requests` , clamped under the 55 s wall, only ever *lowering* the configured budget. The reserve is asymmetric because the failures are: an expired *turn* concedes one server-chosen move (`:451-453`), an empty *bank* is `forfeitPlayer(..., 'lost due to inactivity.')` (`:455`). The unit is a **request**, not a turn — a post-KO replacement is its own request off the same bank (`:286`), and that search had no deadline at all until 2026-08-04. Full derivation, the measured wall-clock distribution, the flags and the R6 validation spec are in [SEARCH.md](#).

Sources

- [Monte Carlo Tree Search Variants for Simultaneous Move Games — Lanctot et al., CIG 2014](#)
- [MCTS in Simultaneous Move Games — Tak, Lanctot & Winands](#)
- [Are Strong Policies Also Good Playout Policies? — AIIDE](#)
- [An Analysis of Monte Carlo Tree Search](#)
- [Fixed-Budget Best-Arm Identification in Structured Bandits](#)
- [On-line Policy Improvement using Monte-Carlo Search — Tesauro & Galperin](#)
- [VGC-Bench: Towards Mastering Diverse Team Strategies in Competitive Pokémon](#)